Instalación y cierre

Guía para preparar Cordova, crear un proyecto y revisar Android, iOS, problemas frecuentes y el cierre documental.

Node.js y npm

Cordova CLI se instala con npm, el gestor de paquetes que acompaña a Node.js. Para la familia Cordova CLI 13.x, la documentación pide Node.js 20.17.0 o posterior.

01

Instala una versión compatible de Node.js

Descarga e instala Node.js desde su sitio oficial o mediante el método de administración de software que use tu sistema. npm se instala junto con Node.js.

02

Consulta las versiones disponibles

Abre una terminal nueva y usa estos comandos. Son una consulta: no instalan ni modifican nada.

Comandos

```
node --version
npm --version
```

Salida ilustrativa

```
v20.17.0
10.x.x
```

Cordova CLI

La CLI es el programa de terminal que crea y organiza un proyecto Cordova. No es la plataforma Android ni la plataforma iOS: esas se agregan después, desde el proyecto.

01

Instala la CLI globalmente

La opción `-g` instala el paquete para poder usar el comando `cordova` desde cualquier carpeta del equipo.

Comando

```
npm install -g cordova
```

02

Consulta la versión de la CLI

Este comando solo muestra la versión disponible en la terminal. Una salida concreta no prueba que Android, iOS o un proyecto ya estén configurados.

Comando

```
cordova --version
```

Salida ilustrativa

```
13.0.0
```

Crear el proyecto

El comando de creación reúne en una carpeta el código web y la configuración que Cordova usará más adelante para preparar plataformas nativas.

01

Elige carpeta, identificador y nombre visible

La carpeta puede llamarse `quever`. El identificador debe ser único para la aplicación, con formato de dominio invertido; `com.ejemplo.quever` es un ejemplo que debe sustituirse antes de distribuir una app.

02

Crea la estructura inicial

Comando

```
cordova create quever com.ejemplo.quever QueVer
```

`cordova create` inicia el proyecto; `quever` es la carpeta; `com.ejemplo.quever` es el identificador; y `QueVer` es el nombre que verá la persona usuaria.

03

Entra a la carpeta creada

Comando

```
cd quever
```

Desde esa carpeta se agregarán plataformas o plugins en secciones posteriores del manual.

Alcance de esta página. Crear el proyecto no instala Android ni iOS, no compila una aplicación y no confirma que un plugin funcione.

Archivos que conviene reconocer

- `www/`: código web de la aplicación, como HTML, CSS, JavaScript e imágenes.
- `config.xml`: configuración de Cordova, incluido el identificador, el nombre y preferencias del proyecto.
- `package.json`: dependencias y metadatos del proyecto.
- `.gitignore`: reglas para indicar qué archivos no se incluyen en el repositorio, si la plantilla lo crea.

Preparar Android

Esta sección reúne el entorno de Android para la familia `cordova-android` 15.x. La tabla oficial publicada actualmente detalla 15.0.x: JDK 17, API 24–36, Build Tools 36.x y Gradle 8.14.2; el anuncio de 15.1.0 no sustituye esa tabla técnica. Son requisitos de referencia, no una prueba de que este equipo o sus plugins sean compatibles.

Antes de elegir sistema

- Instala **JDK 17**, no solo un JRE. La ruta de esa instalación se guardará en `JAVA_HOME`.
- Instala Android Studio. En su primer inicio permite instalar el Android SDK y conserva la ubicación que muestra el asistente.
- En **Tools > SDK Manager**, instala la plataforma Android más alta admitida por la tabla 15.0.x (API 36), **Android SDK Build-Tools 36.x**, **Android SDK Platform-Tools** y **Android SDK Command-line Tools (latest)**. El emulador es opcional y no se configura en este manual.

macOS

01

Define JDK y SDK en el perfil de zsh

En macOS actual, agrega estas líneas a `~/.zprofile`. La primera obtiene la ruta del JDK 17 instalado; la segunda usa la ruta habitual del SDK de Android Studio. Ajusta la ruta si SDK Manager muestra otra.

Contenido para `~/.zprofile`

```
export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/cmdline-
tools/latest/bin:$ANDROID_HOME/build-tools:$ANDROID_HOME/emulator"
```

02

Abre una terminal nueva y consulta las rutas

Estos comandos solo muestran las variables ya definidas. Corrigen el error de escribir `sdk echo $ANDROID_HOME`: `sdk` no forma parte de ese comando.

Comandos de consulta

```
echo "$JAVA_HOME"
echo "$ANDROID_HOME"
```

03

Acepta las licencias del SDK

Cuando las herramientas de línea de comandos ya estén instaladas y disponibles en `PATH`, revisa cada licencia y responde solo si estás de acuerdo con ella.

Comando

```
sdkmanager --licenses
```

Windows

01

Define las variables desde el panel del sistema

Busca **Editar las variables de entorno del sistema**, abre **Variables de entorno** y crea `JAVA_HOME` con la carpeta del JDK 17 (sin `bin`) y `ANDROID_HOME` con la carpeta que muestra SDK Manager. No copies estas rutas sin confirmar tu nombre de usuario y ubicación reales.

Ejemplos de valores

```
JAVA_HOME=C:\Program Files\Java\jdk-17
ANDROID_HOME=C:\Users\tu_usuario\AppData\Local\Android\Sdk
```

02

Agrega herramientas a `Path`

En la misma ventana, edita `Path` y agrega una entrada por cada carpeta bajo `ANDROID_HOME`.

Entradas para `Path`

```
%ANDROID_HOME%\platform-tools
%ANDROID_HOME%\cmdline-tools\latest\bin
%ANDROID_HOME%\build-tools
%ANDROID_HOME%\emulator
```

03

Abre una terminal nueva, consulta y acepta licencias

En Símbolo del sistema, los porcentajes muestran las variables. Después de confirmar que la ruta no está vacía, el administrador del SDK presenta las licencias pendientes.

Comandos en Símbolo del sistema

```
echo %JAVA_HOME%
echo %ANDROID_HOME%
sdkmanager --licenses
```

Gradle: qué instalar y qué no editar

La guía de Cordova requiere un Gradle del sistema. En Windows, agrega también la carpeta `bin` de esa instalación a `Path`; en macOS, sigue el método de instalación indicado por Gradle y asegúrate de que su ejecutable esté disponible en la terminal. Para la línea 15.0.x, Cordova declara Gradle 8.14.2 para el proyecto: el Gradle del sistema crea el Gradle Wrapper, que descarga y usa la versión del proyecto. Por eso ambas versiones no tienen que coincidir.

> **Conserva los valores administrados por Cordova.** No cambies manualmente los archivos generados dentro de `platforms/android` para forzar API, Build Tools o Gradle: pueden regenerarse. Si en el futuro se necesita una propiedad de Gradle, se documentará como configuración del proyecto, no como cambio a esta instalación base.

Preparar iOS

Para trabajar con iOS se necesita una Mac y la aplicación completa Xcode; Windows y Linux no pueden preparar este entorno. Para `cordova-ios` 8.x, la matriz oficial indica iOS 13 o posterior, Xcode 15 o posterior, Node.js 20.17.0 o posterior y CocoaPods 1.16.0 o posterior. Son mínimos de Cordova, no requisitos vigentes para publicar en App Store.

Qué instalar

01

Instala macOS compatible y Xcode completo

Descarga Xcode desde App Store o desde Apple Developer. Una instalación completa aporta `xcodebuild`, el SDK de iOS y el simulador; las Command Line Tools instaladas por separado no sustituyen a Xcode para este propósito. La versión mínima de Xcode para Cordova 8.x es 15, aunque una versión de macOS puede exigir una versión concreta de Xcode.

02

Completa el primer inicio y selecciona Xcode activo

Abre Xcode una vez y acepta su licencia para que instale sus componentes. Si hay varias copias de Xcode o las herramientas apuntan a las Command Line Tools, selecciona la aplicación completa como directorio activo. El comando siguiente es una instrucción para Terminal; no se ejecuta desde este manual.

Seleccionar Xcode instalado

```
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
```

Después se puede confirmar qué ruta está activa con `xcode-select --print-path`. Si falta el componente de línea de comandos, Xcode puede ofrecerlo al primer inicio; `xcode-select --install` solicita solo esas herramientas y no reemplaza la instalación completa de Xcode.

03

Instala el runtime del simulador que vayas a usar

Abre **Xcode > Settings > Platforms** y descarga un runtime de simulador de iOS compatible con tu Xcode. El SDK llega con Xcode, pero los runtimes del simulador se descargan por separado y pueden ocupar bastante espacio. Elegir o iniciar un simulador y agregar la plataforma iOS al proyecto se explica después; esta sección solo deja preparado el entorno.

04

Instala CocoaPods para dependencias nativas

CocoaPods resuelve e integra bibliotecas nativas de iOS que pueden necesitar Cordova o sus plugins. La matriz de `cordova-ios` 8.x exige CocoaPods 1.16.0 o posterior. Si usas Homebrew, este es el método que muestra la guía de Cordova:

Comando

```
brew install cocoapods
```

Una vez instalado, `pod --version` permite consultar la versión. La guía conserva un párrafo antiguo que menciona 1.8.0: para la familia 8.x se toma como referencia el mínimo 1.16.0 de su matriz versionada.

> **No mezcles dos políticas distintas.** Los mínimos anteriores sirven para que Cordova 8.x pueda preparar un proyecto. La distribución en App Store depende además de las reglas actuales de Apple —por ejemplo, el SDK y Xcode que Apple acepte en ese momento—, por lo que deben consultarse antes de enviar una app. Cumplir el mínimo de Cordova no garantiza que una entrega sea aceptada por App Store.

Agregar la plataforma y usar el simulador

01

Agrega iOS desde la carpeta del proyecto

Con el entorno anterior listo y desde la carpeta que contiene `config.xml`, este comando declara la plataforma iOS. La versión se fija para que el ejemplo corresponda a la familia documentada; es texto instructivo y no se ejecuta desde esta página.

Comando

```
cordova platform add ios@8.1.1
```

Al preparar la plataforma, Cordova genera `platforms/ios`, incluido el proyecto y workspace que Xcode abre para revisar la parte nativa. Es contenido generado: evita editarlo como lugar permanente para cambios de la app.

02

Revisa requisitos y prepara una compilación

La CLI puede informar qué herramientas detecta y preparar el proyecto iOS. Los siguientes comandos son referencias para ejecutar en una Mac configurada, no resultados ni comprobaciones realizadas por este manual.

Comandos

```
cordova requirements ios
cordova build ios
```

03

Elige un simulador instalado

En Xcode abre el selector de destino y elige un modelo de iPhone junto con el runtime descargado en **Settings > Platforms**. Después, la instrucción de Cordova para iniciar en el simulador es la siguiente:

Comando

```
cordova run ios --emulator
```

El simulador permite probar la interfaz sin una cuenta de pago ni un iPhone físico, pero no sustituye un dispositivo: hardware, permisos, rendimiento, notificaciones y algunos servicios pueden variar o no estar disponibles.

Usar un iPhone físico y firmar

01

Abre el workspace en Xcode y selecciona el equipo

Conecta el iPhone, abre el archivo `.xcworkspace` generado dentro de `platforms/ios`, selecciona el destino físico y, en **Signing & Capabilities**, elige tu **Team**. Con la firma automática, Xcode relaciona el identificador único de la app (*bundle ID*) con un certificado de desarrollo y un perfil de aprovisionamiento (*provisioning profile*) para permitir instalarla en ese dispositivo.

Puede ser necesario iniciar sesión con la Apple Account en Xcode, confiar en el ordenador y activar Developer Mode en el iPhone cuando iOS lo solicite. Son pasos de seguridad del dispositivo; no son sustituidos por Cordova.

02

Distingue el equipo personal de la membresía de pago

Una Apple Account gratuita puede aparecer como **Personal Team** en Xcode y sirve para firmar y probar personalmente una app en dispositivos propios, sujeto a límites de vigencia y cantidad que Apple puede cambiar. No permite distribuir por App Store ni TestFlight. El Apple Developer Program de pago habilita la distribución y los recursos de cuenta necesarios para publicar; no hace falta para aprender con el simulador.

03

Entiende el papel limitado de `ios-deploy`

`ios-deploy` es una utilidad de línea de comandos que Cordova puede usar para instalar o lanzar una compilación ya firmada en un dispositivo físico; no inicia ni controla el simulador y no reemplaza la firma, el perfil ni Xcode. La matriz de `cordova-ios` 8.x pide `ios-deploy` 1.12.2 o posterior, pero su comportamiento puede depender de la versión de iOS/Xcode: para el primer despliegue y los problemas de firma, Xcode es la ruta principal.

Comando para dispositivo (solo referencia)

```
cordova run ios --device
```

> **Los plugins necesitan su propia revisión.** Que Cordova prepare iOS no demuestra que cada plugin funcione en iOS, en el simulador o en un iPhone. Verifica que el plugin declare soporte para la versión de `cordova-ios`, sus dependencias CocoaPods, permisos y claves de privacidad de Apple; prueba en un dispositivo cuando use hardware. En particular, este manual no ha confirmado la compatibilidad de `cordova-sqlite-storage` 7 con `cordova-ios` 8 ni su funcionamiento.

Requisitos por capa

Las versiones de la CLI, de cada plataforma y del sistema operativo son cosas distintas. "Mínimo" expresa un requisito publicado, no una garantía de compatibilidad de cualquier plugin.

Versiones y herramientas separadas por responsabilidad

Capa	Versión de referencia	Requisitos publicados	Alcance y observaciones
Cordova CLI	13.0.0	Node.js ≥ 20.17.0; npm llega con Node.js.	Git es opcional y solo hace falta si npm o la CLI descargan recursos desde una URL Git.
Cordova Android	15.1.0 (22 jul 2026)	JDK 17; Android SDK; Android Studio; Gradle del sistema; variables `JAVA_HOME` y `ANDROID_HOME`.	La guía oficial vigente detalla 15.0.x: API 24–36, Build Tools ^36.0.0, Gradle 8.14.2, AGP 8.10.1 y Node.js ≥20.17.0 o ≥22.9.0. Esta sección aplica esa referencia a la familia 15.x y advierte que la tabla no enumera 15.1.x de forma separada.
Cordova iOS	8.1.1 (7 jul 2026)	macOS; iOS ≥13; Xcode/xcodebuild ≥15; Node.js ≥20.17.0; CocoaPods ≥1.16.0; ios-deploy ≥1.12.2.	La matriz oficial aplica estos mínimos a toda la familia 8.x. iOS no puede prepararse desde Windows o Linux.

> **Dato que requiere cuidado.** `sdk echo $ANDROID_HOME` no es un comando válido. Una ruta local o la versión de `adb` tampoco demuestran compatibilidad. SQLite es opcional y la compatibilidad de su versión 7 no se ha comprobado aquí.

Diagnóstico sin suposiciones

Cuando un paso no sale como se esperaba, identifica primero la capa que falta: terminal y Node.js, herramientas de plataforma, proyecto o plugin. Los comandos de esta sección son referencias para diagnosticar en tu propio equipo; este manual no los ejecutó ni presenta salidas reales.

Consultas orientativas desde la carpeta del proyecto

```
cordova --version
cordova platform ls
cordova plugin ls
cordova requirements android
cordova requirements ios
```

- **La terminal no reconoce `cordova`.** Confirma que Node.js y npm están disponibles en una terminal nueva; después revisa la instalación global de la CLI y la configuración de rutas de tu sistema. No confundas este caso con un problema del SDK de Android o Xcode.
- **Faltan Java, SDK o licencias en Android.** Revisa `JAVA_HOME`, `ANDROID_HOME` y las entradas de `Path`/`PATH` de la sección Android. Instala los componentes desde SDK Manager y revisa las licencias con `sdkmanager --licenses` solo si aceptas sus términos.
- **Gradle o la API de Android no coinciden.** Vuelve a la matriz de la plataforma que uses y evita cambiar archivos de `platforms/android` a mano: son generados y pueden reemplazarse. Una versión local de `adb` no prueba que el conjunto sea compatible.
- **iOS no encuentra Xcode, CocoaPods o un simulador.** Comprueba que Xcode completo sea el directorio activo, que el runtime esté descargado y que CocoaPods alcance el mínimo documentado. En un iPhone físico, revisa Team, bundle ID, firma y provisioning en Xcode antes de atribuir el problema a Cordova.

- **Un plugin falla al preparar o compilar.** Revisa primero su documentación, versión, dependencias nativas, permisos y claves de privacidad. Que la plataforma se agregue no confirma que el plugin sea compatible; aísla el cambio y consulta el registro de la herramienta que estés usando.

SQLite: complemento opcional

La nota de curso original propone `cordova-sqlite-storage` para dar a la app una API JavaScript de base de datos local. No es un requisito para crear, preparar ni compilar un proyecto Cordova.

01

Agrega el plugin solo si la app necesita datos locales

Desde la carpeta del proyecto, la instrucción de la nota es la siguiente. Antes de usarla, consulta la documentación del plugin para confirmar versiones, plataformas, licencias, permisos y dependencias nativas.

Comando de referencia

```
cordova plugin add cordova-sqlite-storage
```

02

Usa la API cuando Cordova ya esté listo

El plugin expone `window.sqlitePlugin` para abrir y consultar una base local. El código de la aplicación debe esperar el evento de dispositivo listo y manejar errores; no debe asumir que la API existe en un navegador común.

03

Trátalo como una compatibilidad por verificar

La carpeta de ejemplo declara `cordova-sqlite-storage` 7, pero esta guía no instaló, compiló ni probó el plugin. En particular, no confirma su funcionamiento con `cordova-ios` 8, simulador o iPhone; valida el caso real en el entorno que corresponda antes de depender de él.

Checklist final

Úsala antes de continuar hacia una compilación o una entrega. Marcar un punto significa que lo revisaste en tu entorno, no que este manual lo haya comprobado.

- Node.js cumple el mínimo de la CLI y npm está disponible.
- La CLI de Cordova está instalada y el proyecto tiene `www/`, `config.xml` y `package.json`.
- Si se trabajará en Android: JDK 17, Android Studio, SDK, variables, Build Tools, Gradle y licencias se revisaron contra la matriz.

- Si se trabajará en iOS: se usa una Mac con Xcode completo, runtime de simulador, CocoaPods y requisitos de firma revisados.
- Las versiones de CLI, plataforma y sistema se registraron por separado; no se sustituyó la matriz oficial con una salida de `adb`.
- Los plugins necesarios se revisaron de forma individual. SQLite se añadió únicamente si el producto lo requiere.
- Antes de distribuir, se revisarán los requisitos actuales de Google Play o App Store; los mínimos de Cordova no garantizan publicación.

> **Límite de este documento.** Es una guía de preparación y diagnóstico. No se ejecutaron Cordova, la aplicación, compilaciones nativas, emuladores, simuladores ni dispositivos para redactarla.

Fuentes y referencias

- Instalación de la CLI de Cordova — compatibilidad de CLI 13.x y Node.js.
- Guía de Android de Cordova — matriz Android, JDK, SDK, Gradle y variables.
- Guía de iOS de Cordova — matriz iOS 8.x, Xcode, CocoaPods e ios-deploy.
- Cuenta de desarrollador de Apple y comparación de membresías — Personal Team, firma y distribución.
- cordova-sqlite-storage — documentación del plugin SQLite mencionado como opcional; su compatibilidad concreta debe verificarse por proyecto.
- Anuncio de cordova-android 15.1.0.
- Anuncio de cordova-ios 8.1.1.
- **Fuente de partida interna:** `../notasExpo.md`, consultada para recuperar el comando opcional de SQLite; no sustituye la documentación del plugin.

Base documental actualizada el 24 de octubre de 2026. Los comandos se muestran como instrucciones; no se ejecutaron Cordova, la aplicación, compilaciones, emuladores ni dispositivos.